

**INSTITUTO FEDERAL DE CIÊNCIA, EDUCAÇÃO E TECNOLOGIA DO SUL DE
MINAS GERAIS – *CAMPUS* POÇOS DE CALDAS**



TRABALHO DE SISTEMAS DISTRIBUÍDOS

Sistema Distribuído de Pagamentos

Ana Carolina Fuentes
Samuel de Mello Cagnani

Poços de Caldas, agosto de 2026.

1 – Definição e Planejamento

Entrega: 24/08/2026

Tema:

JogaFácil - Uma plataforma distribuída de gerenciamento de espaços esportivos e formação de partidas

1- Descrição

O JogaFácil é uma plataforma distribuída voltada para a conexão entre jogadores, organizadores de partidas e estabelecimentos esportivos, com o objetivo de facilitar a busca, reserva e utilização de quadras esportivas, especialmente campos de futebol society.

A plataforma surge a partir de um problema comum entre jogadores: a dificuldade de encontrar quadras disponíveis em horários convenientes, comparar diferentes estabelecimentos, verificar preços e condições do local e, principalmente, reunir a quantidade necessária de jogadores para realizar uma partida. Do outro lado, proprietários de quadras frequentemente possuem horários ociosos e enfrentam dificuldades para divulgar seus espaços, administrar reservas e alcançar novos clientes.

O JogaFácil busca solucionar esse problema por meio de uma plataforma única, na qual os estabelecimentos podem disponibilizar suas quadras e horários, enquanto os jogadores podem pesquisar locais, criar partidas, encontrar partidas abertas e ocupar vagas disponíveis.

O sistema será desenvolvido como um sistema distribuído baseado em microsserviços, implantado integralmente em infraestrutura de nuvem da Amazon Web Services (AWS). A utilização de uma arquitetura distribuída não será apenas uma decisão tecnológica, mas estará diretamente relacionada às características do problema. A plataforma deverá lidar com múltiplos usuários realizando operações simultaneamente sobre recursos compartilhados, como horários de quadras, reservas e vagas em partidas.

Um dos principais desafios do sistema será garantir que operações concorrentes não provoquem inconsistências. Por exemplo, caso duas pessoas tentem reservar simultaneamente o último horário disponível de uma quadra, o sistema deverá garantir que somente uma das solicitações seja confirmada. Da mesma forma, quando uma partida possuir apenas uma vaga restante e dois jogadores tentarem ocupá-la simultaneamente, apenas uma solicitação deverá ser aceita.

Dessa forma, o JogaFácil será utilizado como cenário para implementação e demonstração dos principais conceitos de Sistemas Distribuídos, incluindo arquitetura distribuída, comunicação entre processos, concorrência, coordenação, sincronização, nomeação, consistência, replicação, tolerância a falhas e segurança.

2. Objetivo geral

Desenvolver uma plataforma distribuída para gerenciamento de quadras esportivas e organização de partidas, utilizando serviços independentes executados em infraestrutura de nuvem, capaz de lidar com operações concorrentes sobre recursos compartilhados e demonstrar, de forma prática, os principais conceitos de Sistemas Distribuídos.

3. Objetivos específicos

O projeto possui os seguintes objetivos específicos:

- Permitir que jogadores realizem cadastro e autenticação na plataforma;
- Permitir que estabelecimentos cadastrem suas quadras e informações relacionadas;
- Permitir o cadastro e gerenciamento de horários disponíveis;
- Permitir que jogadores pesquisem quadras e horários;
- Permitir a criação e gerenciamento de partidas;
- Permitir que jogadores encontrem e participem de partidas abertas;
- Controlar o número máximo de participantes de cada partida;
- Permitir a realização e o cancelamento de reservas;
- Evitar reservas duplicadas para uma mesma quadra e horário;
- Evitar que uma partida ultrapasse o número máximo de participantes;
- Implementar comunicação síncrona entre serviços utilizando REST;
- Implementar comunicação assíncrona utilizando filas e tópicos;
- Implementar mecanismos de coordenação entre instâncias distribuídas;
- Utilizar identificadores únicos para recursos e eventos;
- Implementar mecanismos de replicação e consistência;
- Demonstrar comportamento do sistema diante de falhas;
- Implementar mecanismos de recuperação automática;
- Proteger as comunicações utilizando criptografia;
- Implementar autenticação e autorização;
- Monitorar os componentes distribuídos da aplicação;
- Executar os serviços em containers na infraestrutura AWS.

4. Principais usuários da plataforma

A plataforma será composta inicialmente por três tipos principais de usuários.

4.1 Jogador

O jogador utilizará a plataforma para encontrar locais e partidas para jogar. Entre suas principais

funcionalidades estarão:

- cadastro e login;
- criação e edição do perfil;
- busca de quadras;
- consulta de disponibilidade;
- visualização de partidas abertas;
- criação de partidas;
- participação em partidas;
- cancelamento de participação;
- acompanhamento das partidas das quais participa;
- recebimento de notificações.

Um jogador poderá, por exemplo, procurar por partidas de futebol society próximas, na quinta-feira, às 20h, com até 5 vagas disponíveis. O sistema apresentará as partidas compatíveis com os critérios informados.

4.2 Organizador

O organizador será responsável pela criação e gerenciamento de partidas.

Ele poderá:

- criar uma partida;
- selecionar a quadra;
- definir data e horário;
- definir quantidade máxima de jogadores;
- definir valor por jogador;
- acompanhar os participantes;
- cancelar a partida;
- visualizar o status da partida.

Um exemplo de partida será:

Pelada Quinta-feira Arena Central Quinta-feira, 20:00–21:30 12 de 14 jogadores R\$ 20,00 por jogador 2 vagas disponíveis

4.3 Proprietário do estabelecimento

O proprietário terá acesso a uma área administrativa para gerenciamento do negócio. Entre suas funcionalidades estarão:

- cadastro do estabelecimento;
- cadastro das quadras;
- definição dos horários disponíveis;
- definição de preços;
- gerenciamento das reservas;
- visualização da ocupação das quadras;
- gerenciamento de partidas realizadas no estabelecimento;
- consulta de informações relacionadas às reservas.

Dessa forma, o estabelecimento poderá utilizar a plataforma não apenas para receber reservas, mas também como ferramenta de gerenciamento da operação.

5. Principais recursos do sistema

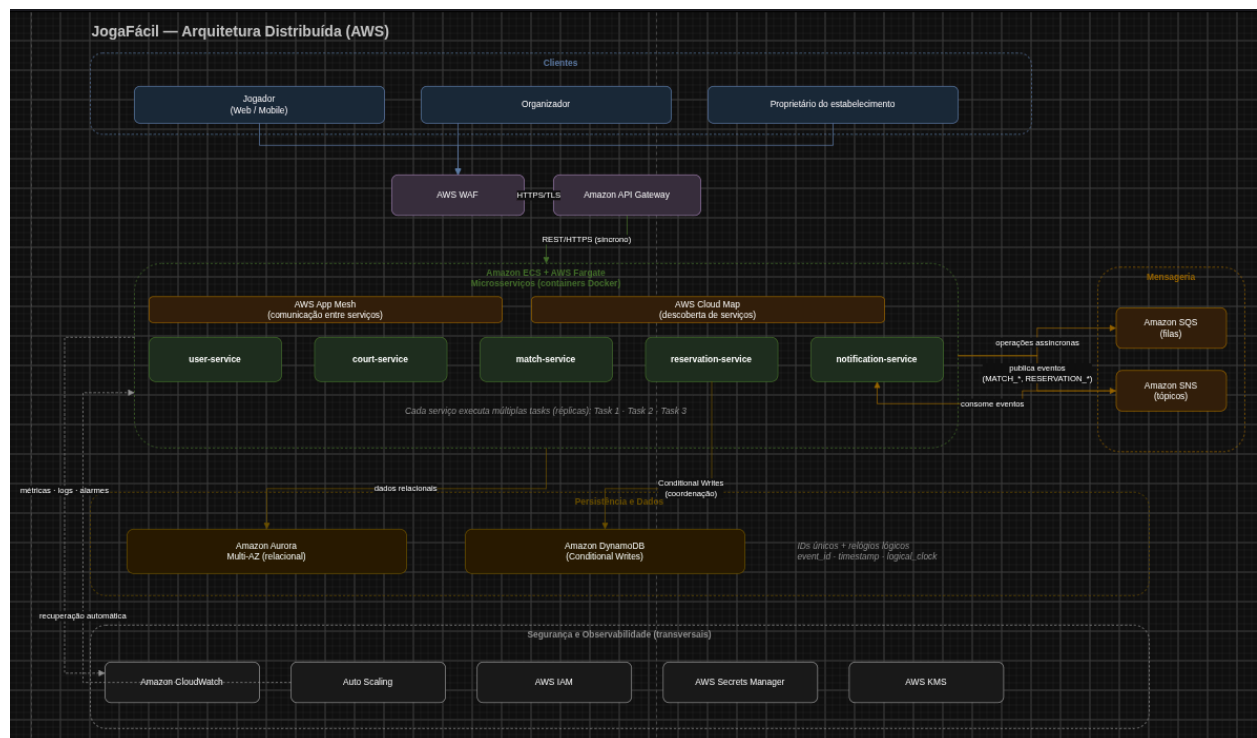
Os principais recursos compartilhados da plataforma serão:

- **Usuários:** Representam jogadores, organizadores e proprietários.
- **Estabelecimentos:** Representam os locais que disponibilizam espaços esportivos.
- **Quadras:** Representam os espaços esportivos que podem ser reservados.
- **Horários:** Representam períodos em que determinada quadra pode ser utilizada.
- **Partidas:** Representam eventos esportivos criados por organizadores.
- **Vagas:** Representam a quantidade de participantes que uma partida suporta.
- **Reservas:** Representam a associação entre um usuário, uma partida ou horário e uma quadra.
- **Eventos:** Representam acontecimentos relevantes do sistema, como criação de partida, entrada de jogador, cancelamento e confirmação de reserva. Esses recursos serão utilizados para demonstrar situações de concorrência, sincronização, consistência e tolerância a falhas.

6. Arquitetura

O JogaFácil adotará uma **arquitetura baseada em microsserviços**, na qual as principais funcionalidades serão divididas em serviços independentes. A arquitetura será executada integralmente na infraestrutura da AWS, utilizando containers e serviços gerenciados.

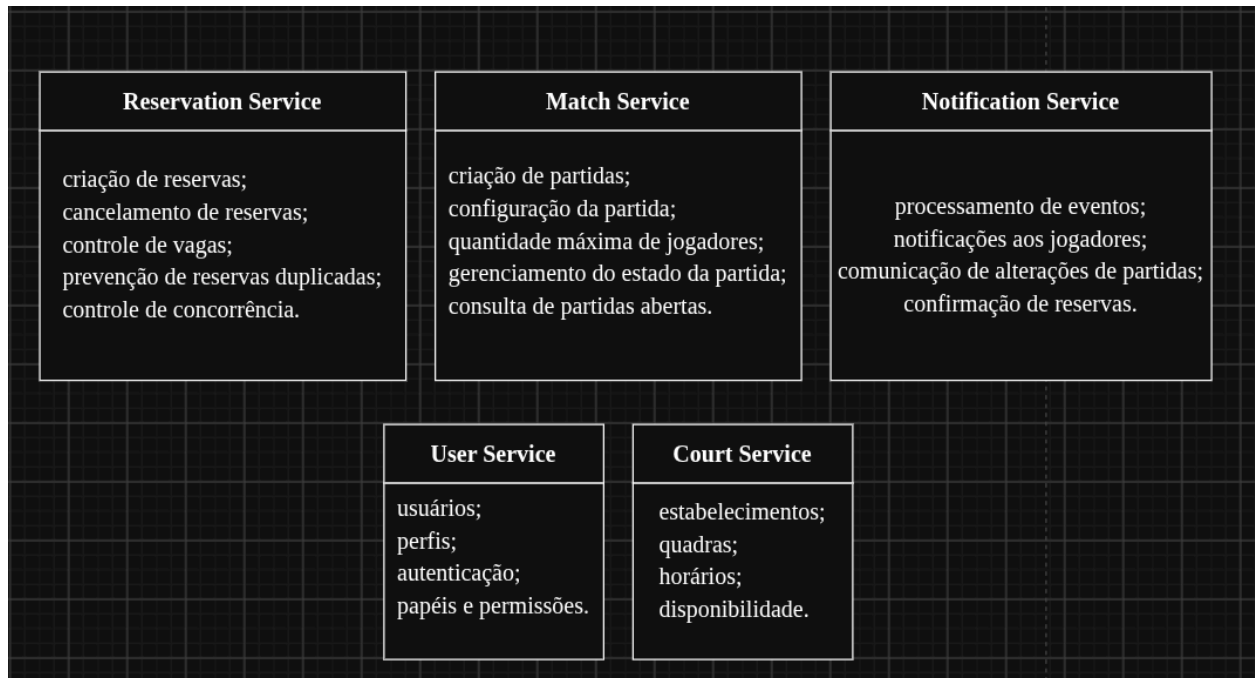
Diagrama inicial de arquitetura:



7. Arquitetura / Middleware

7.1 Estilo arquitetural

O sistema utilizará microsserviços como estilo arquitetural principal. Cada serviço será responsável por um domínio específico e poderá ser executado e escalado independentemente. Os principais serviços serão:



7.2 Middleware

O Amazon API Gateway funcionará como porta de entrada para as APIs da plataforma. A comunicação entre os serviços utilizará:

- REST/HTTPS para operações síncronas;
- Amazon SQS para comunicação assíncrona;
- Amazon SNS para distribuição de eventos;
- AWS Cloud Map para descoberta dos serviços;
- AWS App Mesh como camada de gerenciamento da comunicação interna.

8. Processos e Virtualização

Os serviços serão desenvolvidos como aplicações independentes e empacotados utilizando containers Docker. Os containers serão executados utilizando Amazon ECS com AWS Fargate. A execução poderá possuir múltiplas instâncias do mesmo serviço:

Exemplo:

Reservation Service

- └─ Task 1
- └─ Task 2
- └─ Task 3

Essa estrutura permitirá demonstrar que múltiplos processos/instâncias podem atender requisições simultaneamente.

Concorrência

A concorrência será demonstrada principalmente nas operações de reserva e ocupação de vagas. O sistema deverá garantir que somente uma operação seja confirmada.

9. Comunicação

O projeto utilizará comunicação síncrona e assíncrona.

9.1 REST

A comunicação síncrona será implementada utilizando REST sobre HTTPS.

Exemplos de operações:

- GET /courts
- GET /matches
- GET /matches/{id}
- POST /matches
- POST /reservations
- DELETE /reservations/{id}

9.2 Comunicação orientada a mensagens

O Amazon SQS será utilizado para operações assíncronas. Isso permitirá desacoplar os serviços e evitar que uma operação dependa diretamente da disponibilidade imediata de outro serviço.

9.3 Broadcast e eventos

O Amazon SNS será utilizado para distribuição de eventos. Por exemplo, quando uma partida for completada:

Eventos previstos incluem:

- MATCH_CREATED;
- MATCH_UPDATED;
- MATCH_FULL;

- MATCH_CANCELLED;
- RESERVATION_CREATED;
- RESERVATION_CANCELLED.

Assim, uma mesma ocorrência poderá ser consumida por múltiplos serviços.

A coordenação será necessária porque o sistema possui recursos compartilhados que podem ser acessados simultaneamente. O principal problema será a concorrência em reservas. O projeto utilizará DynamoDB Conditional Writes como mecanismo de controle de concorrência e coordenação. A operação somente será aceita se o estado esperado do recurso ainda for válido.

11. Sincronização e relógios

O sistema também implementará mecanismos para representar a ordenação de eventos distribuídos.

Os eventos importantes poderão possuir:

- identificador único;
- timestamp;
- identificador do serviço;
- contador lógico.

Exemplo:

```
{  
  "event_id": "evt_01",  
  "type": "RESERVATION_CREATED",  
  "match_id": "match_01",  
  "timestamp": "2026-08-25T20:00:00Z",  
  "logical_clock": 42  
}
```

O relógio lógico será utilizado para auxiliar na análise da ordem dos eventos produzidos por diferentes instâncias.

12. Nomeação

O sistema utilizará um esquema de identificação única para todos os principais recursos. Os identificadores serão utilizados para evitar ambiguidades e permitir que os serviços se refiram

aos mesmos recursos de maneira consistente. Além disso, o AWS Cloud Map será utilizado para descoberta dos serviços. Os serviços poderão ser identificados por nomes lógicos, como:

- user-service
- court-service
- match-service
- reservation-service
- notification-service

Dessa forma, os serviços não dependerão de endereços IP fixos.

13. Consistência e Replicação

A plataforma deverá lidar com múltiplas instâncias dos serviços e com operações concorrentes. O estado das principais entidades será armazenado em uma infraestrutura de banco de dados com alta disponibilidade utilizando Amazon Aurora Multi-AZ. Serão realizados testes para analisar diferentes comportamentos de consistência.

Teste de consistência

Será analisado o comportamento do sistema quando diferentes instâncias realizam operações sobre o mesmo recurso.

Serão observados:

- tempo de propagação das atualizações;
- possibilidade de leituras desatualizadas;
- comportamento diante de operações concorrentes;
- comportamento após falhas;
- convergência do estado.

O projeto também permitirá discutir a diferença entre consistência forte e consistência eventual, relacionando o comportamento observado às necessidades do domínio.

14. Tolerância a Falhas

A plataforma será projetada para continuar funcionando mesmo quando uma instância de um serviço apresentar falha. O Amazon CloudWatch será utilizado para monitoramento, métricas e alarmes, enquanto o ambiente de execução poderá realizar a substituição de instâncias que apresentarem falhas. O objetivo será demonstrar:

- detecção de falhas;
- recuperação automática;
- continuidade do serviço;
- preservação do estado;
- comportamento diante de uma operação interrompida.

15. Segurança

A segurança será implementada em diferentes níveis.

15.1 Criptografia da comunicação

Toda comunicação externa será realizada utilizando HTTPS/TLS. As APIs públicas serão protegidas pelo AWS WAF.

15.2 Autenticação

Os usuários precisarão estar autenticados para executar operações protegidas. A plataforma distinguirá diferentes perfis:

- PLAYER
- ORGANIZER
- OWNER
- ADMIN

15.3 Autorização

As permissões serão definidas de acordo com o papel do usuário.

15.4 Segurança da infraestrutura

Serão utilizados:

- AWS IAM para controle de identidades e permissões;
- AWS Secrets Manager para armazenamento de credenciais e informações sensíveis;
- AWS KMS para gerenciamento de chaves criptográficas;
- AWS WAF para proteção das APIs;

→ HTTPS/TLS para comunicação.

16. Observabilidade

O sistema utilizará Amazon CloudWatch para monitoramento da infraestrutura e dos serviços. Serão acompanhadas informações como:

- quantidade de requisições;
- erros;
- tempo de resposta;
- falhas de containers;
- mensagens processadas;
- tamanho das filas;
- operações de reserva;
- ocorrências de concorrência;
- recuperação de instâncias.

Os logs também serão utilizados durante os experimentos para demonstrar o comportamento distribuído do sistema.

17. Definição das tecnologias

Requisito	Tecnologia escolhida	Justificativa / aplicação no projeto
Arquitetura Middleware	Amazon Gateway + API AWS App Mesh	Entrada das APIs e gerenciamento da comunicação entre serviços
Processos e Virtualização	Amazon ECS + AWS Fargate	Execução dos microsserviços em containers
Comunicação	REST/HTTPS + Amazon SQS + Amazon SNS	Comunicação síncrona, processamento assíncrono e distribuição de eventos
Coordenação	Amazon DynamoDB (Conditional Writes)	Controle de concorrência em reservas e vagas
Nomeação	AWS Cloud Map + identificadores	Será utilizado para descoberta e identificação dinâmica dos serviços distribuídos.

	únicos	
Consistência e Replicação	Amazon Aurora Multi-AZ	Persistência e disponibilidade dos dados
Tolerância a Falhas	Amazon CloudWatch + Auto Scaling	Monitoramento, detecção e recuperação
Segurança	AWS IAM + AWS Secrets Manager + AWS KMS + AWS WAF	Autenticação/autorização, segredos, criptografia e proteção da API

18. Escopo do MVP

Para manter o projeto viável durante o semestre, o MVP será concentrado nas funcionalidades essenciais.

- Será implementado
- cadastro e autenticação de usuários;
- perfis de usuário;
- cadastro de estabelecimentos;
- cadastro de quadras;
- cadastro de horários;
- criação de partidas;
- consulta de partidas;
- entrada em partidas;
- controle de vagas;
- reservas;
- cancelamentos;
- notificações;
- comunicação REST;
- comunicação assíncrona;
- concorrência;
- coordenação;
- identificação de recursos;
- replicação;
- testes de consistência;
- tolerância a falhas;
- autenticação e autorização;

→ monitoramento.

Funcionalidades que poderão ser adicionadas posteriormente

- pagamentos reais;
- chat entre jogadores;
- ranking;
- avaliação avançada;
- matchmaking por nível;
- campeonatos;
- estatísticas esportivas;
- aplicativo mobile nativo;
- sistema de fidelidade.

Essas funcionalidades não serão necessárias para demonstrar os requisitos fundamentais de Sistemas Distribuídos.

19. Divisão de responsabilidades

Integrante	Responsabilidades
Integrante 1 – Serviços de Negócio, Concorrência e Dados	• Desenvolvimento e implementação das APIs dos serviços centrais: Reservation Service, Match Service e Court Service. • Modelagem do banco de dados relacional (Amazon Aurora Multi-AZ) e definição dos esquemas de dados. • Implementação dos mecanismos de controle de concorrência e coordenação distribuída com DynamoDB Conditional Writes para

	prevenção de reservas/vagas duplicadas. • Implementação da sincronização e ordenação de eventos distribuídos (timestamps e relógios lógicos). • Realização e validação dos testes de concorrência e consistência de dados.
Integrante 2 – Infraestrutura Cloud, Mensageria, Segurança e Observabilidade	• Configuração e provisionamento da infraestrutura na AWS via containers Docker (Amazon ECS com AWS Fargate). • Configuração do Amazon API Gateway, descoberta de serviços com AWS Cloud Map e malha com AWS App Mesh. • Configuração do pipeline de comunicação assíncrona e publicação de eventos com Amazon SQS e Amazon SNS. • Desenvolvimento do User Service (autenticação, papéis e permissões) e do Notification Service (consumo de tópicos/filas e disparo de notificações). • Implementação das políticas de segurança (AWS IAM, Secrets Manager, KMS, WAF). • Configuração da observabilidade, alarmes e testes de tolerância a falhas via Amazon CloudWatch e Auto Scaling.
Todos os integrantes	• Definição e evolução contínua da arquitetura de microsserviços distribuídos. • Integração dos microsserviços com a infraestrutura cloud e testes ponta a ponta (E2E). • Revisão de código (Code Review) e padronização das APIs. • Elaboração da documentação técnica e relatórios das entregas. • Preparação do ambiente e realização da demonstração/apresentação final do sistema.

Link do Github:

<https://github.com/SamuelCagnani/jogafacil-distributed-system>

